Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Memory Management and Protection **3**

1.1 Context and Goals . . . 3
1.2 Where You Start . . . 3
1.2.1 The DEMO Switch . . . 4
1.3 Unit 1: Memory Management – the Ideas . . . 4
1.3.1 Why an Operating System Manages Memory . . . 4
1.3.2 What the Starting Code Does . . . 5
1.3.3 Static versus Dynamic Allocation . . . 6
1.3.4 The Memory-Management Spectrum . . . 6
1.3.5 A Rogue Task . . . 7
1.3.6 Paging: Pages, Frames and the MMU . . . 7
1.3.7 The TLB and the Cost of Paging . . . 8
1.3.8 Page Faults and Virtual Memory . . . 9
1.3.9 MPU versus MMU . . . 9
1.3.10 A Bad Pointer . . . 10
1.3.11 End of Unit 1 . . . 11
1.4 Unit 2: Task Partitions . . . 11
1.4.1 The Target: One Slice of RAM per Task . . . 11
1.4.2 Linker Tools You Need . . . 12
1.4.3 The Partition Block . . . 13
1.4.4 Checking Where Everything Landed . . . 14
1.4.5 `partition_init`: Giving the Variables Their Values . . . 14
1.4.6 Two Stack Pointers: MSP and PSP . . . 15
1.4.7 Entering a Task: `task_launch` and Its Frame . . . 15
1.4.8 `SVC_Handler` and `svc_dispatch` . . . 16
1.4.9 How a Task Ends: `task_exit` and `sys_exit` . . . 17
1.4.10 The Two Stacks in gdb . . . 18
1.4.11 The Rogue Task, Partitioned . . . 18
1.4.12 End of Unit 2 . . . 18
1.5 Unit 3: The MPU . . . 19
1.5.1 The Cortex-M3 MPU: Regions . . . 19
1.5.2 Permissions: AP, XN and PRIVDEFENA . . . 19
1.5.3 The MPU Registers . . . 20
1.5.4 This Week's Two Regions . . . 20
1.5.5 Computing SIZE, RBAR and RASR . . . 21
1.5.6 Helpers and Prepared Calls . . . 21
1.5.7 Writing `mpu_init` and `mpu_set_task_region` . . . 22
1.5.8 Catching Violations: The MemManage Fault . . . 23

1.5.9 Running with the MPU On 24
1.5.10 Why the Kernel Must Check Pointers 24
1.5.11 How a Task Ends: `task_exit_current` 25
1.5.12 End of Unit 3 26
1.6 Where to Find the Code 27

Chapter 1

Memory Management and Protection

1.1 Context and Goals

At the end of Week 3 the kernel was small but working: unprivileged code reached the hardware only through system calls (`svc`). **Memory**, however, was completely open. The kernel's variables, the application's variables and every stack lived side by side in the same RAM, and unprivileged mode, which blocks certain instructions and the system registers, does not check ordinary loads and stores. The question of this week is therefore: **who may touch which memory?** The answer comes in three steps:

1. **Unit 1: Memory management – the ideas.** Why an operating system manages memory, static versus dynamic allocation, and the spectrum from flat memory to paging and virtual memory (page tables, TLB, page faults, effective access time), ending with MMU versus MPU. No code: you observe the starting code and see what it fails to protect.

2. **Unit 2: Task partitions.** Each task gets its own 4 KB slice of RAM, placed by the linker, with its stack at the bottom. The kernel starts a task on its own **Process Stack Pointer (PSP)** and ends it cleanly.

3. **Unit 3: The MPU.** You program the Cortex-M3 **Memory Protection Unit**, so that the hardware stops any task that accesses memory outside its partition, and the kernel checks every pointer that a task passes to it.

1.2 Where You Start

The code is in two sibling directories, `week-04/base/` and `week-04/solution/`, each with its own `Makefile`. You start in `base/` and turn it step by step into `solution/`. Unless otherwise specified, all paths in these notes are relative to `week-04/solution/`; QEMU runs with `-nographic` and is stopped with `Ctrl-C`.

Unlike Week 3, the base **builds and runs from the first minute**. It already runs two unprivileged tasks, Task A and then Task B, each with its own stack, but without any partitioning or protection. The long parts that are not the focus of this week are already prepared:

- the task table and the code that starts and ends a task (`kernel/task.c`);
- an `SVC_Handler` that finds the system-call frame on either the MSP or the PSP, and the new system call `sys_exit`;
- the CMSIS definitions of the MPU registers and of `MMFAR` (`include/cmsdk.h`);
- the two applications (`src/task_a.c`, `src/task_b.c`) and the `DEMO` switch of the `Makefile`.

What you write is about sixty lines: in **Unit 2** the partition block of the linker script (`scripts/mps2_m3.ld`); in **Unit 3** the two MPU functions (`kernel/mpu.c`), the address report in `MemManage_Handler` (`kernel/faults.c`) and the pointer check in `sys_write_impl` (`kernel/syscall.c`).

1.2.1 The DEMO Switch

`make run DEMO=`*name* builds Task B with a block of deliberate misbehavior, included through a preprocessor test such as `#if defined(DEMO_ROGUE)` in `src/task_b.c`. Each demo is built in its own directory (for example `build/demo-rogue/`), so switching between demos never needs a `make clean`; a plain `make run` builds the normal program in `build/`. The three demos are:

- `rogue`: B writes directly into the kernel's uptime counter;
- `badptr`: B passes a kernel address to `sys_write`;
- `overflow`: B recurses until its stack grows out of its own memory.

Running them on the base, after the partitions and after the MPU shows what each layer of protection can and cannot prevent. Note that a **correct** program prints the same output at every stage: what changes is what a **wrong** or malicious program is allowed to do.

1.3 Unit 1: Memory Management – the Ideas

1.3.1 Why an Operating System Manages Memory

Memory is one shared resource that several programs want at once: the kernel and every task. An operating system has four jobs with respect to memory:

- **Placement.** Decide where each program's code, data and stack go, so that they do not overlap. Two programs that both think they own `0x20000004` would destroy each other's data.
- **Sharing.** Let programs use the same memory when they should: one copy of a library serves every program, a buffer can be handed to a device, two programs can exchange data through a common area.
- **Protection.** Stop a program from reading or writing memory that is not its own, by mistake (a bad pointer, a stack that grows too far) or on purpose.

- **More programs than memory.** When the programs together need more than the RAM available, keep in RAM only the parts in use and the rest somewhere else.

In this unit you write no code. You look at the starting code and see which jobs it does. The short answer: only the first, and only partly.

1.3.2 What the Starting Code Does

Placement. In `week-04/base/`, build the program and ask binutils where the linker put four variables: `uptime_ms` and `tx_buf` (kernel), `a_lines` (Task A) and `b_seconds` (Task B):

```
make
arm-none-eabi-nm -n build/scratch-os.axf | grep -E '\b(uptime_ms|tx_buf|a_
```

```
20000004 B uptime_ms
20000008 b tx_buf
20000054 b a_lines
20000058 b b_seconds
```

The letter is the symbol type: `B`/`b` means `.bss` (upper case for global, lower case for `static`). The space before the names and the `$` after them keep only whole names; otherwise `systick_uptime_ms` would match too. All four variables are packed together just above `0x20000000`, and there is **no border** between kernel and task data: the linker placed them file after file, with the generic `*(.bss*)` rule. If Task B computed `0x20000054` and wrote there, it would change A's variable and nobody would notice. The starting code does placement only, per variable and not per task.

Behavior. `make run` (watch about six seconds, then `Ctrl-C`) prints:

```
scratch-os: kernel started
starting task A
A: hello from task A
A: uptime 0 ms (line 1)
A: uptime 1000 ms (line 2)
A: uptime 2000 ms (line 3)
task A exited
starting task B
B: hello from task B
B: uptime 4000 ms (second 1)
B: uptime 5000 ms (second 2)
B: uptime 6000 ms (second 3)
```

A runs to its end, then B starts: the two tasks **never run at the same time**. B's first line says `4000 ms` because the uptime is the kernel's clock since boot: B starts at about 3000 ms, after A's three waits, and prints one second later. The same output is printed by the finished program: a correct program does not change, only a wrong one does.

The board's memory. In QEMU's monitor, `info mtree -f` lists the memories. What the linker script calls FLASH is `mps.ssram1` at `0x00000000`, 4 MiB; what it

calls RAM is `mps.ssram23` at `0x20000000`, 4 MiB; both also appear at `+0x400000` (an **alias**: the same bytes at a second address). The list says every memory is just `ram`: nothing says who may access it. The bus answers any load or store from any code, privileged or not, so protection must come from somewhere else.

1.3.3 Static versus Dynamic Allocation

The other half of placement is **when** addresses are decided.

- **Static allocation** fixes every address and size at **link time**. The map file shows the whole layout before the program runs and no holes appear, because nothing is freed. The price is flexibility: sizes cannot change at run time.
- **Dynamic allocation** hands out memory at **run time** from a **heap** (`malloc`/`free`). It is flexible, but the layout depends on the order of the calls. After many allocations and frees, free memory is split into small **holes**: with 4 free blocks in total but no hole of 3, a request for 3 blocks **fails** although memory is free (**external fragmentation**).

For an embedded system dynamic allocation has two more problems: it can **fail at run time**, where nobody can read the error, and the time of a `malloc` depends on how many blocks it must search, so it is **not constant**, which is bad for deadlines. This course allocates **statically**; every partition of Unit 2 is fixed at link time.

1.3.4 The Memory-Management Spectrum

Memory management methods can be lined up by asking two questions: is the address changed before it reaches memory (**translation**), and does the hardware stop forbidden accesses (**protection**)?

- **Flat.** Everything shares all of RAM, with no translation and no protection. The starting code is here.
- **Static partitions.** Each task gets a fixed slice of RAM at link time: better placement, but still no protection (Unit 2).
- **MPU.** Hardware checks every access against a short list of allowed regions: protection without translation (Unit 3).
- **Paging with an MMU.** Hardware maps every address to a different physical one: translation and protection together (theory only).
- **Virtual memory.** Paging plus the disk, so that programs can use more memory than the RAM has (theory only).

Each step needs more hardware and buys more flexibility.

1.3.5 A Rogue Task

This is what "no protection" means in practice. With `make run DEMO=rogue`, right after its greeting, Task B executes this block of `src/task_b.c`:

```
extern volatile uint32_t uptime_ms;      /* defined in another file; the l
printf( "B: writing to the kernel's uptime counter\n" );
uptime_ms = 0x12345678;                  /* a store into kernel memory */
```

```
B: writing to the kernel's uptime counter
B: uptime 305420896 ms (second 1)
B: uptime 305421896 ms (second 2)
```

`0x12345678` is 305 419 896 in decimal and the kernel keeps counting from there, so one second later B reads 305 420 896 ms: Task B has **changed the kernel's clock**, for everyone. Nothing stopped it, because unprivileged mode blocks only some **instructions** (for example `msr` to CONTROL) and the system registers, not **ordinary loads and stores** to RAM. The lesson: a bug in one task can silently break the kernel or another task, and the damage shows up far from its cause. To stop it, the hardware must check every access.

1.3.6 Paging: Pages, Frames and the MMU

Paging is used by large operating systems (Linux, Windows, macOS). You do not build it, but you need it to understand what a microcontroller does differently.

A program uses **virtual addresses** (VA); the memory chips see **physical addresses** (PA). Paging maps one to the other:

- The virtual space is cut into equal blocks called **pages** (often 4 KiB); physical memory is cut into **frames** of the same size. Any page can go in any frame.
- A **page table** says which frame holds each page, one table per **process**. Each entry has a **valid** bit (is the page in a frame at all?) and permission bits (read, write, execute, user or kernel only).
- The **MMU** (Memory Management Unit) is the hardware that translates **every** access and checks the permissions. It must be hardware: doing it in software for every load, store and fetch would be far too slow.

An address splits into a **page number** and an **offset**. Only the page number is translated; the offset is copied unchanged. With 4 KiB pages the offset is the low **12 bits** ($2^{12} = 4096$).

A two-level walk. With a 32-bit VA, one flat table indexed by 20 bits would need 2^{20} entries, that is 4 MiB **per process** even for a program using three pages. Real systems use **two levels**: a **page directory** whose entries point to second-level tables, with a register **PTBR** holding the directory address. The address is split into 10 bits of directory index, 10 bits of table index and 12 bits of offset, with 4-byte entries. Translate `VA = 0x00403007`:

```
0x00403007 = 0000000001 | 0000000011 | 000000000111
               dir = 1      tbl = 3      offset = 7
```

1. Read directory entry 1, at `PTBR + 1 × 4`; say it holds the second-level table at `0x00021000`.
2. Read entry 3 of that table, at `0x00021000 + 3 × 4 = 0x0002100C`; say it holds frame `0x0009A000`, valid.
3. PA = frame + offset = `0x0009A000 + 0x007 = 0x0009A007`.

Two levels pay off because a second-level table exists **only where memory is used**: a small process needs one directory (4 KiB) and a few tables instead of a 4 MiB table made mostly of invalid entries.

1.3.7 The TLB and the Cost of Paging

The walk costs **two extra memory reads** per access, which would make memory three times slower. The **TLB** (Translation Lookaside Buffer) is a small, fast cache inside the MMU that keeps recent translations (page number → frame, plus permissions). On a **hit** the frame comes straight from the TLB; on a **miss** the MMU walks the tables and stores the result. Programs reuse the same few pages (loop code, stack, current array), so most accesses hit. Since it is a cache, the OS must **flush** the stale entries when it changes a page table or switches process, otherwise a process could reach the previous one's frames.

Effective access time (EAT). It is the average time of one access, translation included. With `m` the time of one memory access and `h` the **hit ratio**, a hit costs `m` and, with a two-level table, a miss costs `3m` (directory read, table read, the access itself):

$$\text{EAT} = \text{h} * \text{m} + (1 - \text{h}) * 3\text{m} = \text{m} * (3 - 2\text{h})$$

Each miss adds `2m`, so the **slowdown** over plain memory is $2 \times (1 - h)$: 1% of misses makes memory 2% slower. With `m = 10 ns` (a hit costs 10 ns, a miss 30 ns):

no TLB	EAT = 30 ns	3x slower
h = 0.80	EAT = 0.80 * 10 + 0.20 * 30 = 14.0 ns	+40%
h = 0.99	EAT = 0.99 * 10 + 0.01 * 30 = 10.2 ns	+2%

The misses, not the hits, set the cost, so `h` must be close to 1. Useful rules:

- **How high must `h` be?** For a slowdown of at most `s`: $2 \times (1 - h) \leq s$, so $h \geq 1 - s/2$ (at most 5% needs $h \geq 0.975$).
- **Deeper tables.** 64-bit systems use **four** levels: a miss costs `5m` and the slowdown is $4 \times (1 - h)$, twice as much for the same `h`.
- **Why `h` is high: locality.** A loop reading an array misses once per 4 KiB page and hits on the other 1023 words of that page.
- **TLB reach.** 64 entries of 4 KiB cover 64 × 4 KiB = **256 KiB**; a program that jumps at random over 1 GiB misses almost always. Bigger pages (2 MiB) increase the reach.
- **After a flush** the TLB is empty and the first accesses miss: for a real-time task this is **jitter**, since access time depends on what the TLB happens to hold.

Exercise. With `m = 50 ns` and a two-level table: with no TLB the EAT is 150 ns (3x slower); with `h = 0.90` it is $0.90 \times 50 + 0.10 \times 150 = 60$ ns (+20%); with `h = 0.98` it is 52 ns (+4%); for a slowdown of at most 10% we need $h \geq 0.95$. With four levels and `h = 0.98` a miss costs 250 ns, so EAT $= 0.98 \times 50 + 0.02 \times 250 = 54$ ns (+8%). A loop that reads a 64 KiB array of 32-bit words once, with 4 KiB pages and an empty TLB, takes 16 misses out of 16 384 accesses, so $h = 1 - 16/16\,384 \approx 0.999$.

1.3.8 Page Faults and Virtual Memory

If the walk finds an entry with the valid bit clear, or the access breaks its permissions (a write to a read-only page, user code touching a kernel page), the MMU raises a **page fault**, an exception that records the address wanted:

1. If the address is not part of the program or the access is not allowed, the OS **kills the process**.
2. If the address is legal but the page is not in RAM, the OS finds a free frame, reads the page from disk, fills the table entry and sets the valid bit.
3. The handler returns and the hardware **restarts the faulting instruction**. It is restarted, not skipped, because its load or store never happened; restarting makes the fault invisible to the program.

Cost. A fault that reads the disk costs **milliseconds**, against nanoseconds for an access. With `p` the page-fault rate, EAT $= (1 - p) \times m + p \times$ (service time). With `m = 200 ns` and a service time of 8 ms: $p = 1/1000$ gives about 8.2 μs, **forty times slower**; to lose at most 10%, $p \leq 2.5 \times 10^{-6}$, fewer than **one access in 400 000**. Virtual memory works only because locality makes faults so rare, and one fault stops the program for milliseconds at an unpredictable moment: a real-time system cannot accept that.

Virtual memory is paging plus the disk, and it gives two things:

- **Isolation.** Each process has its own page table, so the same virtual address lands in different frames for different processes. A process cannot even **name** another process's memory: its own table translates every address into its own frames, or into a page fault.
- **More than RAM.** Pages not in use wait on disk and a page fault brings them back, so programs together can use more memory than the RAM has, as long as the pages in use at any moment fit.

This answers the fourth job of the OS, and protection in the strongest way, but it needs an MMU, page tables in RAM and a disk.

1.3.9 MPU versus MMU

A microcontroller has neither page tables nor a disk. The Cortex-M3 has an **MPU** (Memory Protection Unit): a short list of **regions**, each with a start address, a size and **permissions** (who may read, write, execute, separately for privileged and unprivileged code).

- On **every** load, store and fetch the MPU checks the address against the regions. If allowed, the access goes on at the **same** address; otherwise it is stopped (the store writes nothing, the load reads nothing) and the CPU raises a **MemManage** fault.
- There is **no translation**: every task sees real physical addresses, and the MPU only answers "may this code touch this address?"

Microcontroller cores (Cortex-M) have an MPU; application cores (Cortex-A, in phones and Linux boards) have an MMU, which turns VA into PA or raises a page fault. The MPU of this board is **off** after reset, which is why the rogue write went through; you turn it on in Unit 3.

Why does a microcontroller not have an MMU?

- **Cost and area:** an MMU needs page tables in RAM, a hardware walker and a TLB; an MPU is a handful of registers.
- **Power:** no walks and no extra reads on a miss.
- **Determinism:** a TLB miss makes an access slower by an unpredictable amount; an MPU check takes the same time every time, which matters for deadlines.
- **No disk** to swap to, and **small, fixed programs** for which regions decided at build time are enough.

What you give up is translation: every task must be placed at its own real address by the linker (Unit 2), and the MPU has only a few regions with rules on size and alignment (Unit 3).

1.3.10 A Bad Pointer

The last demo shows a different leak. With `DEMO=badptr`, Task B asks the **kernel** to touch kernel memory, by passing a kernel address to the system call of Week 3 ("send these `len` bytes from `buf` to the console"):

```
int r = sys_write( (const char *) &uptime_ms, 16 );
printf( "\nB: sys_write returned %d\n", r );
```

```
B: hello from task B
^K^@^@line 2)
A: u
B: sys_write returned 16
```

The 16 bytes start at `uptime_ms`: its 4 bytes hold `0x00000BB8` (3000 ms), stored low byte first (`B8 0B 00 00`), not printable, so the terminal shows odd symbols (`^K^@^@`). The next 12 bytes are the start of `tx_buf`, still holding old text of Task A. Task B has read the kernel's private data **through the kernel**: `sys_write` returns 16 because the kernel, running in the SVC handler, **privileged**, reaches any address it is given.

The consequence for Unit 3: an MPU that restricts only unprivileged code would **not** stop this leak, because the reads are the kernel's. The kernel must check the pointer itself before using it.

1.3.11 End of Unit 1

- An OS manages memory to **place** it, **share** it, **protect** it, and to run more programs than fit. The starting code does placement only, per variable, with no border between kernel and tasks.
- This course allocates **statically**, at link time: dynamic allocation fragments the heap, can fail at run time and takes variable time.
- **Paging:** the MMU translates page to frame through page tables, the TLB caches translations, a page fault restarts the instruction once the page is in. **Virtual memory** adds isolation and the disk.
- **Cost of paging:** EAT $= h \times m + (1 - h) \times 3m$ with two levels; `h` must be close to 1 and page faults extremely rare.
- An **MPU** checks every access against regions, with no translation: cheap, low-power, predictable. The Cortex-M3 has one, switched off. Without it, `DEMO=rogue` changes the kernel's clock and `DEMO=badptr` leaks kernel bytes through a system call.

1.4 Unit 2: Task Partitions

1.4.1 The Target: One Slice of RAM per Task

A **partition** is a slice of RAM that belongs to a single task and holds only its stack and variables. FLASH holds the code, the read-only data and the initial values of the task variables. The goal is this RAM layout:

```
0x20000000   Task A partition (4 KB): [ stack 1 KB | A's variables | padd
0x20001000   Task B partition (4 KB): [ stack 1 KB | B's variables | padd
0x20002000   kernel .data and .bss
   ...       (free RAM)
   top       kernel stack
```

The partitions follow five rules:

- **Fixed.** The linker chooses them at link time (static allocation of Unit 1): no task is created, grown or moved at run time.
- **4 KB, aligned to 4 KB.** The size is a power of two and each start is a multiple of 4096 (`0x20000000`, `0x20001000`), which is exactly the shape the MPU of Unit 3 can protect. The price is waste: a task that needs 1.1 KB still gets 4 KB.
- **RAM only.** Code and read-only data (`.text`, `.rodata`, string literals) stay in FLASH, **shared** by the kernel and both tasks: one copy of `printf` serves everybody.
- **Stack at the bottom.** The stack takes the lowest 1 KB and the variables sit above it. Since the stack grows **down**, an overflow leaves the partition immediately, below its start, where Unit 3 will make it fault. With the stack

at the top, an overflow would silently overwrite the task's own variables, and no protection hardware could tell that write from a legitimate one.

- **The kernel keeps the rest:** its `.data` and `.bss` come after the partitions and its stack stays at the top of RAM.

1.4.2 Linker Tools You Need

An **output section** has two addresses: the **run address** (VMA), where the code uses it, and the **load address** (LMA), where its bytes are stored in the image. They are usually equal, but not for `.data`. The partition block uses:

```
.task_a_data : { ... } > RAM AT > FLASH          /* runs in RAM, initial
_task_a_data_load = LOADADDR(.task_a_data);      /* the FLASH address of
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : { ... } > RAM
*task_a.o(.data .data.* .bss .bss.* COMMON)      /* these input sections,
```

- `> RAM AT > FLASH` (as for Week 2's `.data`): variables run in RAM but their initial bytes are stored in FLASH, which keeps its content without power. Somebody must copy them before use: `Reset_Handler` does it for the kernel; for a partition the kernel does it just before the task's first run.

- `(NOLOAD)` reserves addresses but stores **nothing** in the image: right for a stack or padding.

- `ALIGN(x, 4096)` rounds `x` **up** to a multiple of 4096; before the colon it sets the section's start. For the first partition we use `ALIGN(ORIGIN(RAM), 4096)` because right after `.text` the location counter `.` is still a FLASH address; for B, `.` is already in RAM, so `ALIGN(4096)` is enough.

- `. = start + 4096` jumps to the end of the partition, whatever was placed before.

- **File patterns.** `*task_a.o(...)` takes the variables of `src/task_a.c` only, so the source needs no special marking. `.bss` is put in the same loaded section as `.data`: its zeros are stored in FLASH too (4 bytes here), and a single copy loop initializes every task variable.

- **The first match wins.** Every input section is placed by the first rule that matches it, so the partition rules must come **before** the kernel's generic `*(.data*)` and `*(.bss*)`, otherwise the task variables would stay in kernel RAM.

- **Exported symbols and `ASSERT`** (Week 2). The prepared kernel already uses six symbols per task (`_task_X_part_start`, `_part_end`, `_stack_top`, `_data_start`, `_data_end`, `_data_load`, with `X` = `a` or `b`); your edit only **repoints** them. `ASSERT(cond, "msg")` fails the link if `cond` is false, so a task that grows too big is caught at build time.

1.4.3 The Partition Block

All edits are in `scripts/mps2_m3.ld` (in your copy, `week-04/base/scripts/`), three in total:

1. After the line `TASK_STACK_SIZE = 0x400;` add `PART_SIZE = 0x1000;` (4 KB per partition).

2. Near the end, **delete** everything from the comment `/* >> UNIT 2: delete from this comment down to _task_b_data_load. <<` down to the line `_task_b_data_load = LOADADDR(.data);` included: the flat `.task_stacks` section and the empty data symbols. Keep `__StackLimit` and the `ASSERT` after it, which protect the kernel stack.

3. Replace the comment `/* >> UNIT 2: paste the task partitions here, before .data (see the slides). << */`, right after `.text` and before `_sidata = LOADADDR(.data);`, with the partition of A followed by that of B.

Task A's partition is made of three output sections:

```
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : {
    _task_a_part_start = .;              /* partition start = stack bot
    . = . + TASK_STACK_SIZE;             /* reserve the 1 KB stack */
    _task_a_stack_top = .;               /* the stack grows down from h
} > RAM
.task_a_data : {
    . = ALIGN(4);
    _task_a_data_start = .;              /* run address (RAM) */
    *task_a.o(.data .data.* .bss .bss.* COMMON)
    . = ALIGN(4);
    _task_a_data_end = .;
} > RAM AT > FLASH
_task_a_data_load = LOADADDR(.task_a_data);
.task_a_pad (NOLOAD) : {
    . = _task_a_part_start + PART_SIZE;    /* jump to the partition en
    _task_a_part_end = .;
} > RAM
```

`.task_a_stack` starts at the first 4 KB boundary, `0x20000000`, reserves 1 KB and marks the stack top at `0x20000400`. `.task_a_data` collects the variables of `task_a.o`; the two `ALIGN(4)` keep both ends on a word boundary, because the copy loop copies whole 32-bit words. `.task_a_pad` only jumps the location counter to the end of the 4 KB, so the next section starts after A's full partition.

Task B's partition goes right after `.task_a_pad`, still before `_sidata`. It is **identical** with `a` replaced by `b` and with `ALIGN(4096)` instead of `ALIGN(ORIGIN(RAM), 4096)`, followed by the two link-time guards:

```
ASSERT(_task_a_data_end <= _task_a_part_end, "task A data does not fi
ASSERT(_task_b_data_end <= _task_b_part_end, "task B data does not fi
```

1.4.4 Checking Where Everything Landed

```
make
arm-none-eabi-nm -n build/scratch-os.axf | grep -E '_(_task_|a_lines|b_se

00000dd0 A _task_a_data_load
20000000 B _task_a_part_start
20000400 B _task_a_stack_top
20000400 d a_lines
20001000 B _task_a_part_end
20001000 B _task_b_part_start
20001400 B _task_b_stack_top
20001400 d b_seconds
20002000 B _task_b_part_end
20002004 B uptime_ms
```

(Output abridged: the `_data_start`/`_data_end` symbols and `_task_b_data_load` are omitted.) A's partition is `0x20000000`–`0x20001000` and B's is `0x20001000`–`0x20002000`: 4 KB each, on a 4 KB boundary, with the stack in the bottom 1 KB. `uptime_ms` moved from `0x20000004` to `0x20002004`, because the kernel's data now follow the partitions.

`a_lines` and `_task_a_stack_top` share the address `0x20000400`, but this is not an overlap: the stack top is an **exclusive** bound. The Cortex-M stack is *full descending* (a push first decrements `sp` by 4, then stores), so with `sp = 0x20000400` the first push writes at `0x200003FC` and `0x20000400` itself stays free for the first variable. The same holds at `0x20001400`. In the map file, the sections of `task_a.o` appear under `.task_a_data`: the generic `*(.data*)` found nothing left of that file, because the partition rule came first (**first match wins**).

1.4.5 `partition_init`: Giving the Variables Their Values

The task variables run in RAM but start out in FLASH, so someone must copy them. This is `partition_init` in `kernel/task.c`:

```
void partition_init( const struct task *t )
{
    const uint32_t *src = t->data_load;      /* initial values, in FLASH *
    uint32_t *dst = t->data_start;           /* the variables, in RAM */
    while ( dst < t->data_end )
        *dst++ = *src++;                     /* copy one word, move both
}
```

`struct task` (`kernel/task.h`) holds what the kernel knows about a task: name, entry function, stack top, partition bounds and the three data addresses; the table `tasks` fills it from the linker symbols, declared as arrays (for example `extern uint32_t _task_a_stack_top[];`) because only their address matters. It is the same loop as `Reset_Handler`'s copy of the kernel's `.data`, run once per task by `task_launch`. A task's `.bss` needs no zeroing loop, because its zeros are stored inside the loaded section. In the base the loop copied **nothing**: the flat layout set `_data_start` equal to `_data_end`, which let the prepared code run unchanged from the first minute.

1.4.6 Two Stack Pointers: MSP and PSP

The Cortex-M3 has two stack pointers; `sp` (`r13`) is the one in use, and `mrs`/`msr` can name each directly.

- **MSP** (main): always used in **handler mode**, and in thread mode after reset. It is the kernel's stack, at the top of RAM.
- **PSP** (process): used in thread mode when **SPSEL** (CONTROL bit 1) is 1. Each task's stack is a PSP stack inside its partition.
- **nPRIV** (CONTROL bit 0): when 1, thread mode is unprivileged. A task runs with **CONTROL = 3** (nPRIV = 1, SPSEL = 1).
- **EXC_RETURN** is the value in `lr` that ends an exception: `0xFFFFFFF9` returns to thread mode on MSP, `0xFFFFFFFD` to thread mode on PSP (the hardware sets SPSEL accordingly). On exception **entry** the hardware puts an EXC_RETURN in `lr`, and its **bit 2** tells which stack holds the pushed frame: 1 = PSP, 0 = MSP.

The stack must be **inside the partition**: it holds locals, return addresses and the exception frames. If it lived in kernel RAM, you could not restrict a task to its own 4 KB without taking its stack away. And since handler mode always uses MSP, the kernel's work (interrupts, system calls) never runs on a task's stack.

1.4.7 Entering a Task: `task_launch` and Its Frame

The kernel cannot simply **call** a task: it is inside the SVC handler (handler mode, privileged, MSP) while the task must run in thread mode, unprivileged, on PSP, and a call keeps mode, privilege and stack of the caller. The only mechanism that changes all three at once is the **exception return**: the CPU pops 8 words (r0–r3, r12, lr, pc, xPSR) from the stack selected by EXC_RETURN and resumes at the popped pc. So the kernel **fakes an interruption**: it writes on the task's stack the frame the hardware would have pushed just before the task's first instruction, points PSP at it, and returns with `0xFFFFFFFD`.

`main` (privileged, on MSP) starts everything with one system call, number 5 (`SYS_START_TASKS`): `__asm volatile ( "svc #5" ... )`, followed by `kprintf( "all tasks finished\n" )`. `svc_dispatch` calls `task_start_first`, which calls `task_launch( 0 )`, inside the exception, privileged and on MSP. It does three things.

1. Bookkeeping. It records the running task, prints a message, prepares the MPU (empty in the base) and copies the variables:

```
current = i;
kprintf( "starting task %s\n", t->name );
mpu_set_task_region( t->part_start, t->part_end - t->part_start );
partition_init( t );
```

2. The hand-written frame, at `frame = t->stack_top - 8` (8 words, 32 bytes, below the stack top). For Task A, with stack top `0x20000400`:

Word	Address (A)	Register	Value written
`frame[0..4]`	`0x200003e0`–`0x200003f3`	r0–r3, r12	0
`frame[5]`	`0x200003f4`	lr	`task_exit`
`frame[6]`	`0x200003f8`	pc	`t->entry & ~1`: `task_a`
`frame[7]`	`0x200003fc`	xPSR	`0x01000000` (Thumb bit only)

3. Switch. An `asm` block sets PSP and nPRIV (C cannot name PSP or CONTROL), then the function returns `0xFFFFFFFD`:

```
__asm volatile (
    "msr psp, %0\n"          /* PSP = frame */
    "mrs r1, control\n"      /* r1 = CONTROL */
    "orr r1, r1, #1\n"       /* r1 |= 1: set nPRIV */
    "msr control, r1\n"      /* CONTROL = r1 */
    "isb\n"                  /* make the new CONTROL take effect now */
    : : "r" (frame) : "r1", "memory" );
return EXC_RETURN_THREAD_PSP;                    /* 0xFFFFFFFD */
```

The value goes back through `svc_dispatch` to `SVC_Handler`, the hardware pops the frame and the CPU continues at `task_a`, in thread mode, unprivileged, on PSP. Why it is written this way:

- **Return `0xFFFFFFFD`, not `t->entry()`.** A call would run the task in handler mode, privileged, inside the `svc`, with no protection and a handler that never ends. Only an exception return enters thread mode on PSP unprivileged in one step.
- **PSP and CONTROL after the return.** PSP is `0x20000400` (the 32-byte frame has been popped); CONTROL is **3**: the code sets only nPRIV, the return sets SPSEL. Setting nPRIV inside the handler has no effect until the return, because handler mode is always privileged.
- **xPSR bit 24** is the **T (Thumb) bit**: the Cortex-M runs only Thumb code, and popping T = 0 raises a UsageFault (INVSTATE) at the first instruction. For a similar reason `frame[6]` has bit 0 cleared: the popped pc must be a real instruction address.
- **When `task_a` returns**, it returns through `lr`, popped from `frame[5]`, that is into `task_exit`, which calls `sys_exit`.

1.4.8 `SVC_Handler` and `svc_dispatch`

In Week 3 `SVC_Handler` (`kernel/svc_entry.s`) was two instructions (`mrs r0, msp` and `b svc_dispatch`). Now an `svc` comes from two places: from tasks, on PSP, and once from `main`, on MSP, so the handler must first find which stack holds the frame. It reads **bit 2 of `lr`**: `0xFFFFFFF9` is $\ldots1001_2$ (bit 2 = 0, MSP) and `0xFFFFFFFD` is $\ldots1101_2$ (bit 2 = 1, PSP).

```
tst lr, #4                  /* flags = lr & 4: is bit 2 set? */
ite eq                      /* if ((lr & 4) == 0) ... else ... */
mrseq r0, msp               /*     frame = MSP;  // came from main()
mrsne r0, psp               /* else frame = PSP;  // came from a task
mov r1, lr                  /* exc_return = lr: second argument */
bl svc_dispatch             /* r0 = svc_dispatch(frame, exc_return) *
bx r0                       /* return_from_exception(r0) */
```

- `tst` computes the AND like `ands` but stores nothing, only sets the flags: `tst lr, #4` gives Z = 1 when bit 2 is 0.
- `ite eq` (if-then-else) makes the next two instructions conditional, `eq` (Z = 1) and `ne` (Z = 0): exactly one of the two `mrs` runs, so `r0` holds the right frame.
- `lr` is copied into `r1` **before** `bl`, which overwrites it. The arguments follow the AAPCS: frame in `r0`, EXC_RETURN in `r1`.

When the call comes from `main`, the frame is on MSP and `lr = 0xFFFFFFF9` (measured `msp = 0x203fffd8`, 40 bytes below the top: the 32-byte frame plus 8 bytes pushed by `main`). When it comes from Task A, the frame is on A's PSP inside A's partition and `lr = 0xFFFFFFFD`.

Why Week 3's `b svc_dispatch` no longer works. With `b`, the exception always returned where it came from, but `SYS_START_TASKS` and `SYS_EXIT` must return **somewhere else** (into a task, or back to `main`). So `svc_dispatch( uint32_t *frame, uint32_t exc_return )` now **returns the EXC_RETURN to use**, and the entry ends with `bx r0`, which, with an EXC_RETURN value in `r0`, triggers the exception return. It derives the caller with `from_task = ( exc_return & EXC_RETURN_PSP_BIT ) != 0` and handles three cases:

- `SYS_EXIT` from a task: `return task_exit_current();` starts the next task or goes back to `main` (read in Unit 3);
- `SYS_START_TASKS` from `main`, with no task running: `return task_start_first();` enters Task A. A task that tries number 5 falls through to the table and gets −1 like any unknown number;
- every normal service (`SYS_WRITE`, `SYS_READ`, `SYS_UPTIME`): the result goes into `frame[0]` and `exc_return` is returned unchanged, so the exception goes back to the caller.

This respects the rule of Week 3 that the kernel **never waits inside a handler**: each `svc` ends at once, into whatever should run next. `main`'s `svc #5` "returns" only after the last task has exited, with `0xFFFFFFF9`.

1.4.9 How a Task Ends: `task_exit` and `sys_exit`

The user side is in `lib/syscalls.s`. `sys_exit` is a stub with the new number `SYS_EXIT` (4); since it never returns, it ends with a branch to itself instead of `bx lr`. `task_exit` is the address that `task_launch` puts in `frame[5]`, the task's `lr`:

```
sys_exit:
    svc #4
    b sys_exit
```

```
task_exit:
    bl sys_exit     /* does not come back
    b task_exit     /* never reached */
```

A task can therefore end by calling `sys_exit()` itself, or simply by returning from its function, landing in `task_exit`; Task A uses the second way. The kernel side, `task_exit_current`, is read in Unit 3.

1.4.10 The Two Stacks in gdb

With `make debug`, then `arm-none-eabi-gdb build/scratch-os.axf`, `target remote :1234`, `break *task_a`, `break *task_b`, `continue`, and `p/x $control`, `p/x $psp`, `p/x $msp`, `x/8wx $psp-32` at each stop (the `*` stops at the very first instruction, before any push), you measure:

- **CONTROL = 3** in both tasks (unprivileged, on PSP).
- **PSP** is `0x20000400` in A and `0x20001400` in B: the top of the bottom 1 KB of each partition, so inside its own partition.
- The popped **frame** still shows `lr = 0xb37` (`task_exit` plus 1, the Thumb bit), pc = `task_a` or `task_b`, and `xPSR = 0x01000000`.
- **MSP** is `0x203fffd8` in both: it holds `main`'s waiting frame, and tasks never touch MSP.

1.4.11 The Rogue Task, Partitioned

Running `make run DEMO=rogue` again (`make` rebuilds `build/demo-rogue/`), Task B still stores `0x12345678` into `uptime_ms`, now at `0x20002004`, outside B's partition, and the clock is corrupted **exactly as in Unit 1**. The linker only decided **where** things are: a store to `0x20002004` is an ordinary store, and nothing at run time compares it with B's partition `0x20001000`–`0x20002000`. Partitions **arrange** memory but do not **enforce** anything. What has changed is its shape: each task's memory is now one aligned 4 KB block, so one hardware rule per task is enough to describe what it may touch. That rule is Unit 3.

1.4.12 End of Unit 2

- A **partition** is a fixed slice of RAM for one task, 4 KB and aligned to 4 KB, with the stack at the bottom and the variables above it; code and read-only data stay shared in FLASH.
- The linker builds partitions with `ALIGN(..., 4096)`, `(NOLOAD)` for stack and pad, `> RAM AT > FLASH` for the variables and `*task_a.o(...)` to pick one file. The **first match wins**, so partition rules come before the kernel's, and `ASSERT` checks the size.
- `partition_init` copies each task's variables, zeros included, from FLASH before the first run.

- Tasks run unprivileged on **PSP** with **CONTROL = 3**; the kernel and `main` use **MSP**.
- `task_launch` writes an 8-word frame and returns `0xFFFFFFFD`, the only way into a task. `SVC_Handler` finds the frame with `tst lr, #4` and `svc_dispatch` returns the EXC_RETURN to use.
- `DEMO=rogue` still corrupts the kernel: the linker arranges memory but does not enforce anything.

1.5 Unit 3: The MPU

1.5.1 The Cortex-M3 MPU: Regions

Unit 1 introduced the MPU as a short list of regions checked on every access. On this CPU:

- There are **8 regions**, numbered 0 to 7 (`MPU_TYPE` bits 15:8 hold the count; on this board it reads `0x800`, that is 8).
- Each region has a **base address**, a **size** and **permissions**, and counts only when its ENABLE bit is set.
- The **size** is a power of two of at least 32 bytes, up to 4 GB.
- The **base must be a multiple of the size** (aligned to its size): a 4 KB region can start at `0x20000000` or `0x20001000`, not at `0x20000400`. This is why Unit 2 made each partition 4 KB and aligned to 4 KB: one region covers one partition exactly.
- When an **unprivileged** access falls in no enabled region, or its region refuses it, the access is **stopped** and the CPU raises a MemManage fault.
- For **privileged** code, the bit **PRIVDEFENA** chooses between the same rule and the board's default memory map wherever no region matches.

The MPU is a **core peripheral** at `0xE000ED90` and above, writable only by privileged code, so a task cannot switch it off.

1.5.2 Permissions: AP, XN and PRIVDEFENA

AP (access permissions) is a 3-bit field with one right for privileged and one for unprivileged code:

AP	Privileged	Unprivileged	Used here for
`000`	no access	no access	
`001`	read/write	no access	
`010`	read/write	read-only	
`011`	read/write	read/write	the running task's partition (`MPU_AP_FULL_RW`)
`110`	read-only	read-only	FLASH (`MPU_AP_RO`)

XN (execute never) forbids instruction **fetches** from the region (reads and writes follow AP). A partition holds data and stack, never code, so it gets XN: a task cannot run code it has written into RAM, which closes a classic attack.

PRIVDEFENA is bit 2 of `MPU_CTRL`. When 1, a **privileged** access that hits no region uses the default memory map, so the kernel keeps access to all RAM and peripherals without a region for each, and only the tasks are restricted. When 0, even the kernel may touch only what the regions allow. This week uses PRIVDEFENA = 1: the kernel is trusted and the MPU's purpose is to contain the tasks.

1.5.3 The MPU Registers

Register	Address	Fields used this week
`MPU_TYPE`	`0xE000ED90`	bits 15:8: number of regions (read-only)
`MPU_CTRL`	`0xE000ED94`	bit 0 ENABLE (the MPU checks accesses); bit 2 PRIVDEFENA
`MPU_RNR`	`0xE000ED98`	region number: which region `RBAR`/`RASR` refer to
`MPU_RBAR`	`0xE000ED9C`	bits 31:5 base; bit 4 VALID; bits 3:0 REGION
`MPU_RASR`	`0xE000EDA0`	bit 28 XN; bits 26:24 AP; bits 18:16 S, C, B; bits 5:1 SIZE; bit 0 ENABLE

There is one pair `RBAR`/`RASR` for all eight regions, and `RNR` selects the current one. As a shortcut, writing `RBAR` with **VALID** (bit 4) set also sets the region number from its REGION field, so a single write replaces `RNR` then `RBAR`; this week's code uses it.

The **SIZE** field (5 bits) holds $\log_2(\text{size}) - 1$, so a region has $2^{\text{SIZE}+1}$ bytes; the -1 lets 5 bits reach 4 GB. **S, C, B** (shareable, cacheable, bufferable) describe the memory type for caches and multi-core systems; this CPU has no cache, so they change nothing here (the code sets C for FLASH, S and C for RAM). TEX and SRD stay at 0.

1.5.4 This Week's Two Regions

- **Region 0** covers FLASH: base `0x00000000`, size 4 MB, read-only and executable, the same for every task (code and string constants).

- **Region 1** covers the partition of the task running **now**, read/write and XN. The kernel moves it every time it starts a task.

- **Everything else** has **no region** for unprivileged code: the other task's partition, kernel data and stack, every peripheral. Any access by a task there causes a fault.

The "MPU state" of a task is therefore just one value: the base address of Region 1.

1.5.5 Computing SIZE, RBAR and RASR

SIZE = $\log_2(\text{size}) - 1$; since it sits in bits 5:1 it is multiplied by 2 before ENABLE (bit 0) is added:

- Partition: $4096 = 2^{12}$, SIZE = 11 (`0b01011`); $11 \times 2 =$ `0x16`, with ENABLE `0x17`.
- FLASH: 4 MB = 2^{22}, SIZE = 21 (`0b10101`); $21 \times 2 =$ `0x2A`, with ENABLE `0x2B`.

RBAR = base | VALID (`0x10`) | region number: Region 0 gives `0x00000010`; Region 1 gives `0x20000011` for Task A and `0x20001011` for Task B. The base fills bits 31:5 and a base aligned to a size of at least 32 always ends in five 0 bits, leaving room for VALID and REGION.

RASR of Region 1 for Task B is `0x13060017`:

Bits	Field	Value	Meaning
28	XN	1	no instruction fetch (`0x10000000`)
26:24	AP	`011`	read/write for both (`0x03000000`)
18, 17	S, C	1, 1	shareable, cacheable (`0x00060000`)
5:1	SIZE	11	2^{12} = 4 KB (`0x16`)
0	ENABLE	1	region on (`0x01`)

The sum is `0x10000000 + 0x03000000 + 0x00060000 + 0x16 + 0x01 = 0x13060017`. Region 0 is built the same way, from AP `110` (`0x06000000`), C (`0x00020000`) and `0x2B`, without XN: `0x0602002B`.

Three exercises:

- *A 32 KB region: SIZE and low byte of* ***`RASR`****?* 32 KB = 2^{15}, SIZE = 14, $14 \times 2 =$ `0x1C`, with ENABLE `0x1D`.
- *Could it start at* ***`0x20001000`****?* **No**: the base must be a multiple of 32 KB (`0x8000`); `0x20000000` and `0x20008000` are valid.
- *A task needs 5 KB: which region size?* **8 KB**, the next power of two (3 KB unused), the same price as for Unit 2's partitions.

1.5.6 Helpers and Prepared Calls

- `MPU_RASR_SIZE( n )` (`include/cmsdk.h`) takes $n = \log_2(\text{size})$ and computes `((n) - 1) « 1`: write `MPU_RASR_SIZE( 22 )` for 4 MB and `( 12 )` for 4 KB.
- `__builtin_ctz( x )` (GCC) counts the trailing 0 bits of `x`; for a power of two this is its $\log_2$ (`__builtin_ctz( 4096 )` is 12).
- `dsb` waits until every earlier write has really finished and `isb` then makes the following instructions see the new settings; after writing MPU registers you use both, so no later access is checked against the old regions.

`cmsdk.h` also gives the registers as a `struct` (`MPU->CTRL`, `MPU->RBAR`, `MPU->RASR`, ...) and one macro per field (`MPU_RBAR_VALID`, `MPU_RASR_XN`, `MPU_RASR_AP( v )` with `MPU_AP_RO` and `MPU_AP_FULL_RW`, `MPU_RASR_S`, `_C`, `_ENABLE`, `MPU_CTRL_ENABLE`, `MPU_CTRL_PRIVDEFENA`). The calls are prepared: `main` calls `mpu_init()` once before its `svc #5`, and `task_launch` calls `mpu_set_task_region( part_start, part_end - part_start )` before entering each task. In the base both bodies are empty, so the MPU is off.

1.5.7 Writing `mpu_init` and `mpu_set_task_region`

In `kernel/mpu.c` replace both functions, from `void mpu_init( void )` to the end of the file. `mpu_init` programs Region 0 and switches the MPU on:

```
void mpu_init( void )
{
    /* Region 0 = FLASH: base 0, 4 MB = 2^22 B, read-only, code may run.
    MPU->RBAR = 0x00000000 | MPU_RBAR_VALID | 0;          /* = 0x00000010
    MPU->RASR = MPU_RASR_AP( MPU_AP_RO ) | MPU_RASR_C
              | MPU_RASR_SIZE( 22 ) | MPU_RASR_ENABLE;    /* = 0x0602002B

    /* MPU on. PRIVDEFENA: the kernel keeps the default memory map. */
    MPU->CTRL = MPU_CTRL_ENABLE | MPU_CTRL_PRIVDEFENA;     /* = 5 */
    __asm volatile ( "dsb" ::: "memory" );   /* finish the writes ... */
    __asm volatile ( "isb" ::: "memory" );   /* ... before the next instr
}
```

After reset the MPU is disabled: until `ENABLE` is set in `MPU->CTRL`, no protection is enforced however the regions are programmed. Region 1 is not set yet, but this is not a problem: `main` and the kernel are privileged and keep the default map through `PRIVDEFENA`, and no unprivileged code runs before `task_launch` sets Region 1. The second function is called by `task_launch` every time a task starts, and moves Region 1 onto that task's partition:

```
void mpu_set_task_region( uint32_t base, uint32_t size )
{
    /* log2 of a power of two = its trailing zero bits: 4096 -> 12. */
    uint32_t log2_size = __builtin_ctz( size );

    /* Region 1 = the partition: read-write, never executable (XN). */
    MPU->RBAR = base | MPU_RBAR_VALID | 1;                 /* B: 0x2000101
    MPU->RASR = MPU_RASR_XN | MPU_RASR_AP( MPU_AP_FULL_RW )
              | MPU_RASR_S | MPU_RASR_C
              | MPU_RASR_SIZE( log2_size ) | MPU_RASR_ENABLE; /* 0x130600
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}
```

The function computes SIZE from the size it receives, which must be a power of two with `base` a multiple of it (the linker script guarantees both). The lines `(void) base;` and `(void) size;` of the base disappear: they only silenced "unused

parameter" warnings while the body was empty. `make` now builds with no errors and no warnings.

1.5.8 Catching Violations: The MemManage Fault

When the MPU refuses an access, the access does **not** happen (a store writes nothing, a load reads nothing) and the CPU raises **MemManage**, exception 4, enabled at boot by `MEMFAULTENA` in `SHCSR` (Week 2). `MemManage_Handler` in `kernel/faults.c` runs privileged on the MSP. The low byte of **CFSR** (`0xE000ED28`) tells why:

Bit	Name	Set when
0	`IACCVIOL`	an **instruction fetch** was refused (for example from an XN region)
1	`DACCVIOL`	a **data** load or store was refused
3	`MUNSTKERR`	popping a frame on **exception return** was refused
4	`MSTKERR`	pushing the 8-word frame on **exception entry** (stacking) was refused
7	`MMARVALID`	**MMFAR** holds the refused address

MMFAR (`0xE000ED34`, `SCB->MMFAR`) holds the refused data address. Read it only if `MMARVALID` is 1: for a fetch or stacking fault the CPU does not write it, and it may hold a stale value. Several bits can be set at once.

Reporting the address. In `MemManage_Handler`, replace the line `kprintf( "MemManage: CFSR=0x%x\n", SCB->CFSR );` and the comment `/* >> UNIT 3: report MMFAR and the cause bits here. << */` with the code below (the `halt();` that follows stays):

```
uint32_t cfsr = SCB->CFSR;
kprintf( "MemManage: CFSR=0x%x\n", cfsr );

/* MMFAR holds a real address only when MMARVALID is set. */
if ( cfsr & SCB_CFSR_MMARVALID )
    kprintf( "  address 0x%x\n", SCB->MMFAR );

/* Name the cause. Several bits can be set at once. */
if ( cfsr & SCB_CFSR_DACCVIOL )
    kprintf( "  cause: data access\n" );
if ( cfsr & SCB_CFSR_IACCVIOL )
    kprintf( "  cause: instruction fetch\n" );
if ( cfsr & SCB_CFSR_MSTKERR )
    kprintf( "  cause: stacking\n" );
```

`CFSR` is read once so that every test sees the same value, and the cause tests are separate `if`s rather than an `else if` chain because several bits can be set together. The masks `SCB_CFSR_...` are in `include/cmsdk.h`. `kprintf` works inside the handler because the handler is privileged.

1.5.9 Running with the MPU On

Normal run. `make run` prints the **same lines** as in Units 1 and 2. A and B can still print, although the MPU lets them touch only FLASH and their own partition, because those are the only areas they touch themselves: code and string constants are in FLASH (Region 0), while variables, stack and the `printf` buffer (a local, hence on the task's stack) are in the partition (Region 1). The tasks never touch the UART: `sys_write` and `sys_uptime` run in the privileged kernel, which keeps the default map, and the SysTick and UART handlers are privileged too. While A runs, Region 1 has `RBAR = 0x20000011`; from `starting task B` on it has `0x20001011`; Region 0 stays the same.

`DEMO=rogue`. B's store to `uptime_ms` is refused:

```
B: writing to the kernel's uptime counter
MemManage: CFSR=0x82
  address 0x20002004
  cause: data access
```

`0x82` = `0x80` + `0x02`, that is **MMARVALID** + **DACCVIOL**: a data access was refused and `MMFAR` holds its address. With `arm-none-eabi-nm build/demo-rogue/scratch-os.axf | grep ' uptime_ms$'`, `0x20002004` is the address of `uptime_ms`: the report gives the address of the culprit and `nm` turns it into a name. The store **never happened**, so the clock is intact, and the handler halts the system: the MPU caught the bug at the very instruction that caused it, not later and somewhere else.

`DEMO=overflow`. B calls `eat_stack`, which recurses 4 levels with a 1 KB local array each, needing about 4 KB of stack against B's 1 KB:

```
B: recursing
MemManage: CFSR=0x92
  address 0x20000fdc
  cause: data access
  cause: stacking
```

`0x92` = `0x80` + `0x10` + `0x02`: MMARVALID + **MSTKERR** + DACCVIOL. The address `0x20000fdc` is 36 bytes **below** B's partition (`0x20001000`–`0x20002000`), inside **A's partition**: Region 1 covers only B, so the first store there is refused (DACCVIOL). The **stacking** cause appears because, to enter the handler, the CPU pushes the 8-word frame on the stack in use, the PSP, which also points below the partition: that push is refused too (MSTKERR). The frame is lost, but the handler runs on the MSP, so the report is still correct. This is why Unit 2 put the stack at the **bottom**: an overflow leaves the partition at once and faults at the first store. Without the MPU, the same demo keeps writing into other memory and ends much later in `UsageFault: CFSR=0x20000` (INVSTATE, from a corrupted state popped from the stack), a report far from the real cause.

1.5.10 Why the Kernel Must Check Pointers

`DEMO=badptr` is **not** stopped by the MPU: `sys_write_impl` (`kernel/syscall.c`) runs inside `SVC_Handler`, **privileged**, and `PRIVDEFENA` gives privileged code the whole default map. The kernel must therefore check every pointer a task gives it against what **that task** could access by itself. For `sys_write`, the whole buffer

`[buf, buf + len)` must lie **entirely** inside the caller's partition or **entirely** inside FLASH (`0x00000000`–`0x00400000`, where string constants are); otherwise the call returns −1 and prints nothing. Two prepared helpers:

- `task_current` (`kernel/task.c`) returns the running task's `struct task`, with `part_start` and `part_end`; it returns 0 when no task runs, i.e. the trusted kernel is calling.
- `in_range` (`kernel/syscall.c`) tells whether `[p, p + len)` is fully inside `[start, end)`:

```
static int in_range( uint32_t p, uint32_t len, uint32_t start, uint32_t e
{
    return p >= start && p <= end && len <= end - p;
}
```

Why not `p + len <= end`? Because the sum can **wrap around**: with `p = 0x20001F00` and `len = 0xFFFFFF00` it wraps modulo 2^{32} to `0x20001E00`, which passes, and the kernel would read almost all of memory. Once `p <= end` is known, `end - p` cannot wrap, so `len <= end - p` is always correct.

In `sys_write_impl`, replace the comment `/* >> UNIT 3: check that [buf, buf + len) is allowed (in_range). << */` with:

```
    /* Accept the buffer only inside the caller's partition or inside
     * FLASH (string constants). The kernel itself (no task) is trusted.
    const struct task *t = task_current();
    if ( t != 0
         && !in_range( a0, len, t->part_start, t->part_end )
         && !in_range( a0, len, 0x00000000, 0x00400000 ) )
        return ( uint32_t ) -1;
```

The check runs before `uart_write`, so a refused buffer is never read (`in_range` was marked `__attribute__(( unused ))` in the base to silence a warning; you may leave it). Now `make run DEMO=badptr` prints:

```
B: hello from task B

B: sys_write returned -1
B: uptime 4000 ms (second 1)
```

No kernel bytes appear: `0x20002004` is neither in B's partition nor in FLASH, so `sys_write` returns −1 at once. The empty line comes from the `\n` at the start of the demo's `printf`, and B keeps running, because a refused system call is an error the task can handle, not a fault. B's own `printf` lines still pass, because their buffer is on B's stack, inside its partition. The **MPU** stops what a task does by itself and the **kernel's checks** stop what a task asks the kernel to do: both are needed.

1.5.11 How a Task Ends: `task_exit_current`

`svc_dispatch` calls `task_exit_current` (`kernel/task.c`) for `SYS_EXIT` from a task:

```
uint32_t task_exit_current( void )
{
    kprintf( "task %s exited\n", tasks[current].name );
    if ( current + 1 < ( int ) NUM_TASKS )
        return task_launch( current + 1 );        /* start the next task */

    current = -1;                                 /* no task runs any more
    /* asm block: CONTROL = CONTROL & ~1;  clear nPRIV, then isb */
    return EXC_RETURN_THREAD_MSP;                 /* 0xFFFFFFF9: back to m
}
```

When Task A ends:

1. `task_a` returns through `lr` into `task_exit`, which calls `sys_exit` (`svc #4`). `SVC_Handler` sees bit 2 of `lr` set and picks the PSP, `svc_dispatch` sees `SYS_EXIT` from a task and calls `task_exit_current`, which prints `task A exited` and returns `task_launch( 1 )`: B's frame and `0xFFFFFFFD`. The exception returns straight into Task B; A's old frame is left on its stack.

2. `task_launch` calls `mpu_set_task_region`: Region 1 now covers B's partition (`RBAR = 0x20001011`). A's partition **no longer has** a region, so B can neither read nor write it. The "MPU state" of a task is just Region 1.

3. If B returned too, no task would be left: the kernel sets `current` to -1, clears nPRIV and returns `0xFFFFFFF9` (thread mode on MSP), so `main` continues after its `svc #5`. Clearing nPRIV is essential: `main` runs in thread mode, and with nPRIV still 1 it would run **unprivileged**, its `kprintf` would fault on the UART (no region), and it could not fix this because unprivileged code cannot write `CONTROL` (Week 2).

4. You never see `all tasks finished` because B never returns (its loop is infinite); if it did, `main` would print it and idle in its `wfi` loop.

1.5.12 End of Unit 3

For the tasks it runs, the kernel now does all four jobs of the week:

- **Placement:** each task has a fixed 4 KB partition built by the linker (stack at the bottom, variables above); code and constants are shared in FLASH.
- **Separate stacks:** tasks run unprivileged on the PSP inside their partition, the kernel on the MSP; `task_launch` enters a task by an exception return and `task_exit_current` ends it and starts the next.
- **Protection by the hardware:** the MPU allows a task FLASH (read and execute) and its own partition (read/write, no execute) and nothing else; a refused access raises MemManage, whose report gives cause and exact address (`rogue: 0x82`; `overflow: 0x92`).
- **Protection by the kernel:** `sys_write` refuses any buffer the calling task could not read by itself (`badptr` gets -1), with an overflow-safe `in_range`.

A correct program prints exactly the same output as in Unit 1; what has changed is what a **wrong** one can do.

1.6 Where to Find the Code

The files changed this week, all in `week-04/solution/` (`base/` is the starting point; if these notes and the code disagree, the code is the reference):

- `scripts/mps2_m3.ld`: the two partitions and their `ASSERT`s; `include/cmsdk.h`: MPU registers, field macros, `MMFAR`, CFSR masks.
- `kernel/task.h`, `task.c`: `struct task`, `partition_init`, `task_current`, `task_launch`, `task_start_first`, `task_exit_current`.
- `kernel/svc_entry.s`: `SVC_Handler`; `kernel/syscall.c`: `svc_dispatch`, `in_range`, the checked `sys_write_impl`.
- `kernel/mpu.c`: `mpu_init`, `mpu_set_task_region`; `kernel/faults.c`: `MemManage_Handler`.
- `lib/syscalls.s`, `syscalls.h`: `sys_exit`, `task_exit`, service numbers 4 and 5; `src/main.c`, `task_a.c`, `task_b.c`; `Makefile`: the `DEMO` switch.